Horizon: Dependency-Aware Autonomous Infrastructure Recovery—Architecture, Empirical Evaluation and Patentability Assessment

Sarthak Patil Utkarsh Vidwat Ganesh Beldar Jay Magar

Independent Research / Hackathon Prototype, India

November 2026

Abstract—When distributed infrastructure fails, restarting a single component rarely restores service: faults propagate along dependency chains, and restarting a service before the components it depends on are healthy prolongs the outage. This paper presents Horizon, a dependency-aware recovery platform that maintains a directed graph of infrastructure nodes, computes the blast radius of a failure by reverse traversal, derives a dependency-constrained recovery order with Kahn's algorithm, and suspends high-risk steps until an approval token is supplied. We describe the system exactly as implemented at repository commit 6d459f6 and evaluate the repository's own engine code. On 1,000 random 50-node topologies, Horizon's topological ordering produced 0.00 dependency-order violations per incident, against 4.18 for breadth-first discovery order and 16.02 for alphabetical order, and blast-radius precision and recall were both 1.00. The TypeScript engine ordered a 10,000-node graph in 13.9 ms (median), whereas the Python engine's blast-radius routine scaled quadratically (7.5 s at the same size). A conformance audit of 15 design objectives found 5 fully met, 5 partly met and 5 not yet implemented, including health-verified progression, mid-recovery re-planning and cryptographically verified approval. We relate the work to US 12 556 441 B1 and other prior art, assess patentability conservatively, and outline a roadmap for an Indian provisional filing.

***Index Terms*—autonomous recovery, dependency graph, blast radius, topological sorting, AIOps, human-in-the-loop, self-healing infrastructure, patentability.**

I. Introduction

A. Background and Motivation

Modern enterprise infrastructure is, in effect, a directed graph of dependencies: databases sit beneath application services, services beneath gateways, and gateways beneath user-facing front-ends, while caches, queues and identity providers cut across the layers. When a foundational node fails, impairment propagates along these edges, and the conventional response—locate the failed component and restart it—ignores the order in which components must return. Restarting an application while its data store is still unavailable produces crash loops and alert storms and lengthens the mean time to recovery (MTTR). In practice, recovery still relies on engineers coordinating in incident war rooms with static runbooks that describe yesterday's topology, while monitoring and AIOps tooling [1] is better at surfacing symptoms than at stating what should be restored first.

B. Problem Statement and Research Questions

This paper asks whether a recovery controller that (i) maintains an explicit dependency graph, (ii) computes the blast radius of a failure on that graph, (iii) derives a dependency-constrained recovery order and (iv) pauses high-risk steps for approval is a sound and distinguishable technical design. We restrict the empirical scope to properties that can be measured on the implementation as it exists. The research questions are:

RQ1. Does dependency-constrained ordering reduce dependency-order violations relative to naive orderings?

RQ2. How accurate is the blast-radius computation?

RQ3. How does analysis latency scale with graph size across the three engine implementations?

RQ4. How does the system behave under transient faults and circular dependencies?

RQ5. To what extent does the implementation satisfy its design objectives, and which parts remain distinguishable over prior art?

MTTR reduction against human operators is explicitly out of scope: recovery actions in the prototype are simulated, so any MTTR figure would measure timers rather than recovery (Section VI-A).

C. Contributions

(1) A precise formulation of dependency-ordered recovery with a measurable violation metric (Eq. 3). (2) An open reference implementation in three runtimes—an in-browser TypeScript engine, a Hono edge API and a FastAPI Model Context Protocol (MCP) server—sharing one algorithmic core. (3) A reproducible evaluation that executes the repository's own classes: zero ordering violations versus 4.18–16.02 for naive baselines on 1,000 random topologies, blast-radius precision and recall of 1.00, and ordering of 10,000 nodes in 13.9 ms in TypeScript. (4) A conformance audit of 15 design objectives against the code, a candid account of defects, and a conservative novelty and patentability assessment.

The remainder of the paper is organised as follows. Section II reviews related work, Section III describes the architecture as implemented, Section IV the recovery method, Section V the methodology, Section VI the results, Section VII conformance and limitations, Sections VIII–X novelty, security and the intellectual-property roadmap, and Section XI concludes.

II. Related Work

A. Academic Research

Dependency information has long been used to reduce recovery cost. Dependency-aware rollback in task-based runtimes cancels and recomputes fewer tasks than global rollback when nodes fail, as shown by simulation [2]; the

setting, however, is high-performance checkpoint/restart rather than service infrastructure. Surveys of AIOps for failure management cover the detection, diagnosis and remediation pipeline and the growing use of large language models (LLMs) for root-cause hypotheses [1]. Recent preprints report agentic architectures that plan remediation under ordering constraints [3] and a decision layer combining dependency-aware event correlation, admission gates, bounded remediation and post-action verification [4]. Taken together, these works show that dependency-aware planning and gated automation are active research areas.

B. Commercial Platforms

StackStorm offers event-driven automation through a rules engine and workflows, but remediation is typically expressed in pre-authored packs; Rundeck concentrates on job orchestration with audit and access control, with recovery sequences written as jobs or playbooks [5]. Kubernetes controllers provide restart, rescheduling and readiness/liveness probing at pod and deployment level but keep no cross-service application dependency graph [5]. Enterprise AIOps suites (Dynatrace, Datadog, Splunk, ServiceNow ITOM) are strong in detection and correlation; in our reading of public documentation, autonomous dependency-ordered execution in them is partial and policy-bound [5]. Rubrik Autonomous Business Recovery recovers a minimum viable business with explicit ordering across network, compute and data layers [6].

C. Patent Literature

The closest issued patent we identified is US 12 556 441 B1 [7], whose claims cover monitoring computing devices, generating a map of interconnections among services, infrastructure components and computing resources, detecting a technical issue, and dynamically generating an ordered remediation plan from that map, including ordered remediation according to a detected chain of dependencies. Related documents describe dependency-evaluating remediation [8], ordered restoration derived from a dependency map [9], startup workflows that withhold a component until its dependencies are operational and that remediate circular dependencies [10], recovery segments of co-dependent components [11], and automatic discovery of software relationships [12]. These set a high bar for any claim that merely builds a dependency graph and recovers in topological order. Table I summarises the overlap.

TABLE I
SELECTED PRIOR ART COMPARED WITH HORIZON

Reference	Year	Core mechanism	Dep.	Seq.	Overlap (1–5)
US 12 556 441 B1 [7]	2026	Automated dependency map; dynamically generated ordered remediation plan	Yes	Yes	5 (extremely similar)
US 8 667 096 B2 [9]	—	Dependency map used to derive ordered list of nodes for restoration	Yes	Yes	4 (very high)
US 9 183 092 B1 [10]	—	Dependency graph; startup withheld until dependencies operational; circular-dependency remediation	Yes	Yes	4 (very high)
US 2023/0205657 A1 [8]	2023	Dependency-aware service monitoring and remediation	Yes	Partial	4 (very high)
Rubrik ABR [6]	2026	Sequenced minimum-viable-business recovery	Yes	Yes	4 (very high)
AUTO-OPS [4]	2026	Dependency-aware gates, bounded remediation, verification	Yes	Yes	4 (very high)
StackStorm / Rundeck [5]	2010s	Event-driven rules; job and playbook orchestration	Limited	Static	3 (high, execution)
Kubernetes controllers [5]	2015+	Pod-level restart, rescheduling, probes	Local	No	2 (moderate)
Horizon (this work)	2026	Reverse-BFS blast radius; Kahn levels; risk-gated playbooks	Yes	Partial	—

Note. Dep. = dependency-aware; Seq. = dynamic dependency-ordered sequencing. Overlap is a provisional, author-assigned ordinal (1 = low, 5 = extremely similar) from abstract- and claim-level reading; it is not a legal opinion. “Partial” for Horizon: Kahn levels are computed in the API and MCP engines, but the browser plan generator orders steps by node-type tiers (Section IV-D).

III. System Architecture as Implemented

A. Overview

Horizon is a Bun-workspace monorepo [14]. The front-end uses React 19, Vite 8 and TypeScript and embeds a recovery engine that runs entirely in the browser. A Hono-based edge API (`api/v1`) exposes the same graph algorithms over REST, and a FastAPI MCP server registers 14 tools for external agents over JSON-RPC and server-sent events. Solidity contracts for an audit vault and a subscription token accompany a client for the MST blockchain testnet, and a Sarvam AI client provides natural-language operator commands. State is held in process memory in every engine; the Firebase SDK is initialised for user authentication, but its Realtime Database client is not used to store cluster state at the evaluated commit.

The default simulated cluster has $|V| = 7$ nodes and $|E| = 9$ dependency edges: a PostgreSQL primary and replica, a Redis cache, an authentication service, a payment service, an API gateway and a web front-end. The longest dependency chain (db-primary → redis-cache → auth-service → api-gateway → web-frontend) contains five nodes. The MCP server seeds a different seven-node topology that adds a Kafka broker and a payment worker.

B. Implementation Inventory

Table II lists the components relevant to recovery, their size, and their status. The same three graph operations—cycle detection, blast radius and topological ordering—are implemented independently in TypeScript (browser), TypeScript (API) and Python (MCP), which allows cross-checking but also lets the implementations diverge.

TABLE II
IMPLEMENTATION INVENTORY AND STATUS AT COMMIT 6D459F6

Component	Source file	LOC	Status and notes
Dependency graph (web)	engine/dependencyGraph.ts	201	Implemented: DFS cycle check, Kahn order, BFS blast radius; Kahn order used by tests only
Cluster state / job manager	engine/state.ts	403	Implemented in memory; node-type tiers; 700 ms timer-driven step progression
Watchdog	engine/watchdog.ts	157	Simulated: 4 s tick, no consecutive-miss filter, seeded MTTR history
Graph engine (API)	api/_lib/graph.ts	177	Implemented: cycle path, blast radius, topological levels, critical path
Playbooks	api/_lib/playbooks.ts	131	Implemented: 3 declarative playbooks with per-step low/high risk
Edge API routes	api/v1/[[...route]].ts	330	Implemented; in-memory incidents; pauses on pending high-risk step
Topology engine (MCP)	mcpserver/app/engine/topology.py	960	Implemented: 3-miss probes, Kahn tiers, gated playbooks; O(|V|²) blast radius
Approval verifier (MCP)	mcpserver/app/engine/blockchain.py	123	Format check only (prefix and length); no signature recovery
Chain client / signer (web)	engine/mstBlockchain.ts	594	RPC calls; approval token is a SHA-256-derived string
Audit vault contract	contracts/contracts/HorizonAuditVault.sol	41	Written; not invoked by application code
LLM assistant	engine/sarvamAgent.ts	411	External API; outside the recovery critical path
Docker bridge	engine/dockerBridge.ts	62	Disabled: `isEnabled()` is false; start/stop return false

Note. LOC = physical lines (`wc -l`). Web paths are relative to `apps/web/src`. Repository: https://github.com/Precise-Goals/horizon, commit 6d459f6 [14].

IV. Recovery Method

A. Graph Model and Cycle Detection

Let $G = (V, E)$ be a directed graph in which $(u, v) \in E$ means that node v depends on node u, so u must be healthy before v can be restored. Every engine stores a forward (dependent → provider) and a reverse (provider → dependents) adjacency map. Cycles are detected by depth-first search with a recursion stack in $O(|V| + |E|)$ time.

B. Failure Detection

The MCP engine uses a sliding window: a node is declared down only after three consecutive missed probes, and any successful probe resets its counter. The browser watchdog instead polls in-memory node status every 4 s and spawns a recovery job on the first down or degraded observation; it has no consecutive-miss filter.

C. Blast Radius and Severity

For a failed node f, the blast radius is the set of nodes reachable from f along reverse edges:

$$B(f) = \{\, v \in V \setminus \{f\} : f \rightsquigarrow v \,\} \tag{1}$$

It is computed by breadth-first search over the reverse adjacency map while recording the maximum depth d. The web engine assigns a severity class $\sigma(f)$:

$$\sigma = \text{critical if } |B| \geq 5 \text{ or } d \geq 3;\ \text{high if } |B| \geq 3;\ \text{medium if } |B| \geq 1;\ \text{else low} \tag{2}$$

D. Dependency-Constrained Sequencing

A recovery order π over the affected set $A = \{f\} \cup B(f)$ is valid if and only if every provider precedes its dependents. Horizon obtains π with Kahn's algorithm [13]: nodes with no unresolved providers inside A form level 0; removing them exposes level 1, and so on. The procedure runs in $O(|V| + |E|)$ and stops with an incomplete order exactly when A contains a cycle. The quality of an ordering is measured by the number of violated edges:

$$\Phi(\pi) = |\ \{\ (u, v) \in E : u, v \in A, \mathrm{pos}_\pi(v) \< \mathrm{pos}_\pi(u)\ \}\ | \quad (3)$$

Three realisations exist. The web method `getTopologicalRecoveryOrder` returns a total order; it is covered by unit tests but is not called by the plan generator. The API method `computeTopologicalRecoveryLevels` returns levels over the affected set and attaches them to the incident record. The Python engine computes levels to reject cyclic topologies before recovery, while its steps come from fixed playbooks. In the browser plan generator (`startRecovery`), steps are emitted in fixed tiers—isolation, approval gate, target restoration, downstream caches, downstream applications and gateways, audit anchoring—and, within a tier, in breadth-first discovery order. Nodes of other types (for example a downstream database replica) receive no restoration step. Section VI quantifies the consequences.

E. Risk Classification and Approval Gating

In the web engine, a gate step is inserted after isolation in every incident; it is flagged high-risk and the job enters the `PAUSED_APPROVAL` state until `approveGate` is called. In the MCP engine, a gate is required when the playbook is database failover or restore-from-backup or when the target is a database. In the API, playbook steps carry a low or high risk label and the tick handler refuses to advance past a pending high-risk step. The web approval token is the SHA-256 digest of a JSON object containing the incident identifier, step title, target service, commander address, timestamp and chain identifier, encoded as `0xmst_<32 hex>_<time>`. It is a digest-derived token, not an ECDSA or EIP-712 signature produced by a wallet key, and the MCP verifier checks only its prefix and length.

F. Audit Trail and LLM Assistance

The web engine appends each state transition to an in-memory log capped at 200 entries; the MCP engine keeps timeline events keyed by job and incident. A Solidity audit vault with an owner-only `recordAuditHash` function that rejects duplicates exists, but no application code calls it, and the "Merkle anchoring" step of the web plan is a plan step rather than a transaction. The LLM client handles natural-language commands and diagnostic suggestions only; plan generation involves no network call.

V. Experimental Methodology

A. Setup

All experiments run against repository commit 6d459f6 on a single-vCPU container (Intel Xeon @ 2.10 GHz, 3 GB RAM) with Bun 1.4.4, Node 22.22 and Python 3.12.3. The harness imports and calls the repository's own classes—`DependencyGraph`, `ClusterStateManager`, `DependencyGraphEngine`, `TopologyEngine` and `BlockchainEngine`—rather than re-implementing them. Because the Python models module fails to import at this commit (Section VI-F), the MCP experiments use a scratch copy in which the single missing type alias is defined; no other change is made. All random workloads use fixed seeds, and latencies are medians of repeated runs, so absolute values are indicative of a small machine.

B. Workloads, Baselines and Metrics

W1 is the default seven-node cluster with a failure injected at each node. *W2* consists of random DAGs: node i depends on one to three (ordering, $n = 50$) or one to four (scaling) uniformly chosen earlier nodes, labels are permuted to decouple names from order, and node types are uniform over database, cache, application and gateway. For the ordering experiment, 1,000 incidents are drawn with a target among the first 20 topological positions and retained when $|B| \geq 5$. Five orderings of the affected set are compared: Kahn topological order, alphabetical order, random order, breadth-first discovery order, and the steps produced by the web `startRecovery`. Metrics are Φ (Eq. 3), the percentage of trials with Φ > 0, coverage (the fraction of affected nodes that receive an explicit restoration step), blast-radius precision P and recall R against an independently coded depth-first reachability ground truth, latency, and detection and cycle-handling outcomes.

C. Scope and Threats to Validity

Topologies are synthetic, recovery actions are simulated, no human participates, and no MTTR comparison is attempted. Random DAGs with at most four providers per node may be sparser than production service meshes. Latencies come from one machine. The ordering results therefore establish algorithmic correctness and coverage, not operational benefit.

VI. Results

A. Default Cluster (RQ1, RQ2)

Table III reports a single-node failure at each of the seven nodes. Blast-radius precision and recall equal 1.00 in every case, and no generated restoration order violates a dependency ($\Phi = 0$). Plan generation took 0.51 ms for the first (cold) call and 0.02–0.04 ms afterwards. Two limitations are visible. For db-primary the plan restores five of the six affected nodes, because the downstream database replica is not of a type for which the generator emits a step. And every incident, even a leaf failure such as web-frontend, pauses at the approval gate.

The simulated end-to-end duration of the db-primary job was 5.50 s ($n = 3$: 5503, 5502, 5502 ms), which equals the sum of hard-coded timers (600 ms + 7 × 700 ms) and must not be read as an MTTR measurement. Likewise, the rolling-MTTR histories shown by the watchdog and the MCP engine are initialised with constants (18.2, 22.4, 15.8, 19.1 s and 38.5, 42.0, 48.5, 36.2, 44.0 s respectively), so we do not report them.

TABLE III
SINGLE-FAILURE INJECTION ON THE DEFAULT SEVEN-NODE CLUSTER

Failed node	|B|	d	σ	P	R	Steps	Φ	Restored	Levels	Plan (ms)
db-primary	6	3	critical	1.00	1.00	9	0	5 / 6	5	0.51
db-replica	0	0	low	1.00	1.00	4	0	—	1	0.04
redis-cache	4	2	high	1.00	1.00	8	0	4 / 4	4	0.04
auth-service	3	2	high	1.00	1.00	7	0	3 / 3	3	0.03
api-gateway	1	1	medium	1.00	1.00	5	0	1 / 1	2	0.03
web-frontend	0	0	low	1.00	1.00	4	0	—	1	0.02
payment-service	0	0	low	1.00	1.00	4	0	—	1	0.02

Note. $|B|$ = blast-radius size; d = cascade depth; σ = severity (Eq. 2); P, R = blast-radius precision and recall against an independent ground truth; Steps = steps generated by `startRecovery`, each including exactly one high-risk gate; Φ = ordering violations (Eq. 3); Restored = affected nodes that receive a restoration step; Levels = Kahn levels over $\{f\} \cup B(f)$ from the API engine; Plan = plan-generation time.

B. Ordering Correctness on Random Topologies (RQ1, RQ2)

Table IV summarises 1,000 incidents on random 50-node DAGs with a mean blast radius of 22.3 nodes. Blast-radius precision and recall were 1.00 in all trials. Kahn ordering restricted to the affected set produced no violations. Alphabetical and random orderings violated at least one dependency in 99.7% and 99.9% of trials (about 16 violated edges per incident), and breadth-first discovery order—the order in which the blast radius is found—violated one in 61.5% of trials. The web plan generator produced violations in 81.3% of trials (4.84 per incident) and, because it emits steps only for cache, application and gateway nodes, covered 76.4% of the affected nodes. Its zero-violation result on the default cluster (Table III) therefore reflects the layout of that cluster rather than a guarantee.

TABLE IV
ORDERING STRATEGIES ON 1,000 RANDOM 50-NODE DAGS

Strategy	Mean Φ	Trials with Φ > 0 (%)	Coverage (%)
Kahn topological	**0.00**	**0.0**	100.0
Alphabetical	16.02	99.7	100.0
Random	15.94	99.9	100.0
BFS discovery	4.18	61.5	100.0
Web `startRecovery`	4.84	81.3	76.4

Note. Φ per Eq. (3) over the affected set; coverage = share of affected nodes with an explicit restoration step.

C. Scalability (RQ3)

TypeScript analysis time grows near-linearly (Table V): a tenfold increase from 1,000 to 10,000 nodes raises the median topological-ordering time from 1.00 ms to 13.9 ms. The Python blast-radius routine scans every node for each visited node, so it is quadratic in practice: 0.49 ms at 100 nodes, 49.7 ms at 1,000 and 7,497 ms at 10,000 (a 151-fold increase for a tenfold increase in size). Python's Kahn routine scales acceptably (121 ms at 10,000 nodes). Rebuilding the reverse adjacency map once per query would remove the quadratic term.

TABLE V
MEDIAN ANALYSIS LATENCY VERSUS GRAPH SIZE

Nodes |V|	Edges |E|	TS cycle check (ms)	TS topological order (ms)	TS blast radius (ms)	Py Kahn levels (ms)	Py blast radius (ms)	Reps TS / Py
100	241	0.05	0.17	0.05	0.30	0.49	15 / 15
1,000	2,496	0.25	1.00	0.46	6.27	49.7	15 / 15
5,000	12,257	1.55	7.13	2.27	46.3	1,724	5 / 3
10,000	24,757	3.02	13.9	5.87	121	7,497	5 / 3

Note. TS = TypeScript engine; Py = Python MCP engine. Blast radius is measured for the first topological node (worst case). $|E|$ is reported for the TypeScript workload; Python workloads use the same generator shape (one to four providers per node).

D. Failure Detection and Cycle Handling (RQ4)

Table VI shows that the Python probe window behaves as specified: runs of one or two missed probes never declare a failure and runs of three or more declare it at the third probe. The browser watchdog has no such filter. Cycle detection had no false positives on 1,000 acyclic graphs and found every injected cycle (998 of 998; two of 1,000 injections could not be constructed). However, the web and API ordering routines silently return an incomplete order on a cyclic graph, so safety depends on callers invoking the cycle check first; only the Python engine raises an error, and it rejected all 500 injected cycles. Mounting a cyclic custom pipeline in the Python engine was rejected and the previous topology was restored.

TABLE VI
FAILURE DETECTION AND CYCLE-HANDLING OUTCOMES

Test	Outcome
Py probe: miss runs of 1 or 2, then success	Not declared; healthy
Py probe: miss runs of 3, 4, 5	Declared down at probe 3 (3 / 3 runs)
Browser watchdog: one down flag	Job spawned next tick (code inspection)
Acyclic graphs flagged as cyclic	0 / 1,000
Injected cycle detected, web `hasCycle`	998 / 998
Injected cycle detected, API `detectCycle`	998 / 998
Web order silently truncated on cycle	998 / 998 (no error raised)
API levels silently truncated on cycle	998 / 998 (no error raised)
Py `trigger_recovery` rejects cycle	500 / 500 (ValueError)

Test	Outcome
Py custom pipeline with 2-node cycle	Rejected; topology restored

Note. Py = Python MCP engine. Injection adds one back-edge to a random 50-node DAG.

E. Approval Gating and Token Validation

Table VII shows that the Python verifier accepts any string that begins with 0x and has at least ten characters, including arbitrary text such as 0xzzzzzzzz; three of six test inputs were accepted. Table VIII compares gating behaviour: the web engine pauses at the gate for all seven failure targets, whereas the Python engine gates only the database failover and completes other recoveries immediately. Once a gate is approved, the Python engine marks all steps completed in a single call and enforces a minimum reported recovery time of 18.5 s, so no per-step health verification takes place.

TABLE VII
APPROVAL-TOKEN VALIDATION (PYTHON VERIFIER)

Submitted token	Accepted
“0x” + 64 hex characters	Yes
“0x” + 8 arbitrary characters	Yes
0xdeadbeef (10 characters)	Yes
0x12 (too short)	No
abcdef0123 (no prefix)	No
Empty string	No

TABLE VIII
APPROVAL GATING BY FAILED NODE

Failed node	Web engine	Python engine
db-primary	Pause; 9 steps	Pause; 4 steps, 1 high-risk
redis-cache	Pause; 8 steps	No gate; completed
auth-service	Pause; 7 steps	No gate; completed
api-gateway	Pause; 5 steps	No gate; completed

F. Automated Test Suites

The TypeScript suite passes in full (Table IX). At the evaluated commit the Python suite cannot be collected, because `models.py` uses the undefined name `StepRisk` (line 197), so the MCP server cannot start. Defining the alias as `Literal["low", "medium", "high"]`, consistent with the other step models, allows all 55 tests to pass.

TABLE IX
AUTOMATED TEST RESULTS

Suite	Files	Tests	Pass	Fail	Time (s)
TypeScript (bun test)	21	130	130	0	3.35
Python at HEAD	8	0 run	—	—	—
Python, alias defined	8	55	55	0	0.48

Note. The TypeScript run executed 667 assertions. Python at HEAD fails at collection with a `NameError`.

VII. CONFORMANCE, LIMITATIONS AND PLANNED EVALUATION

A. Conformance to Design Objectives

Table X audits the design objectives of the platform against the code and the measurements above. Five objectives are fully met, five are partly met and five are not met. The partly met and unmet objectives are precisely the ones that distinguish a safe recovery controller from a graph-analysis toolkit, and they define the engineering still required.

TABLE X
DESIGN OBJECTIVES AND IMPLEMENTATION CONFORMANCE

#	Design objective	Status	Evidence ((m) measured, (c) code inspection)
1	Directed dependency graph with cycle detection	Met	0 / 1,000 false positives; 998 / 998 injected cycles detected (m)
2	Blast radius by reverse traversal	Met	P = R = 1.00 on 1,007 incidents (7 default + 1,000 random) (m)
3	Topological recovery order, providers first	Partly met	Kahn Φ = 0 on 1,000 graphs (m); but web generator uses type tiers (Φ > 0 in 81.3% of trials, 76.4% coverage) and Python playbooks are fixed (m, c)
4	Pause before high-risk steps	Met	Web: 7 / 7 incidents enter PAUSED_APPROVAL (m); API tick pauses on pending high-risk step (c); Python gates database failover (m)
5	Per-step risk classification	Partly met	API and Python playbooks label steps low/medium/high (c); web marks only the gate step high-risk (m)
6	Cryptographically signed approval	Partly met	Web token is a SHA-256-derived string (c); Python verifier is format-only, accepts 3 of 6 test inputs (m)
7	Health verification before dependants are touched	Not met	Web steps advance on 700 ms timers (c, m); Python completes all steps in one call (m)
8	Mid-recovery re-planning on a failed check	Not met	No code path re-invokes planning after a job has started (c)
9	Audit record of every transition	Partly met	In-memory log capped at 200 entries (c); Python events in memory; vault contract not invoked (c)
10	Detection resilient to transient faults	Partly met	3-miss rule verified in Python engine (m); absent from browser watchdog (c)
11	Execution on real Docker / Kubernetes targets	Not met	`dockerBridge.isEnabled()` is false; Kubernetes module emits manifests only (c)
12	Shared real-time state in Firebase RTDB	Not met	RTDB client instantiated but unused; Firebase used for authentication (c)
13	Deterministic critical path without LLM dependency	Met	`startRecovery` makes no network or LLM call; Sarvam used for commands and diagnosis (c)
14	MTTR reduction against manual and static baselines	Not met	MTTR histories are seeded constants; README targets (95%, < 120 s) unmeasured (c)
15	Described seven-node simulated cluster	Met	`state.ts` seeds 7 nodes and 9 edges (c, m)

B. Engineering Required to Close the Gaps

Closing the gaps is mostly engineering rather than research: (i) generate plans from Kahn levels over the affected set in `startRecovery` and emit a step for every node type; (ii) add a per-step health probe that gates progression and, on failure, recomputes the plan from the updated graph; (iii) replace the digest token with wallet-signed EIP-712 typed data and verify it server-side with nonce and expiry; (iv) persist audit entries and call the vault contract; (v) rebuild adjacency once in the Python blast-radius routine and define `StepRisk`; (vi) apply the consecutive-miss rule in the browser watchdog; and (vii) replace seeded MTTR values with measured ones.

C. Planned Evaluation

Once real executors and health gates exist, a rigorous comparison should use four conditions: unaided manual recovery from dashboards, a static runbook, Horizon with dependency sequencing disabled (random or alphabetical order), and the full pipeline. Measured quantities should include MTTR, mean time to detect, recovery success rate, dependency-order violations, operator interventions, approval latency and blast-radius precision and recall against injected cascades. Scenarios should cover a single database failure, an API failure, a cache failure, simultaneous multi-node failure, a stale graph, an injected cycle, an approval timeout and an action that fails part-way through a sequence.

VIII. Novelty and Patentability Assessment

This section is a technical assessment only and is not legal advice.

A. Mechanism-Level Novelty

Table XI decomposes Horizon into separable mechanisms. Individually, dependency graphs, reverse-traversal blast radius, topological sorting, risk gates and audit logs are well established. Dynamic generation of an ordered plan from a dependency map (mechanism C) is claimed in US 12 556 441 B1 [7] and echoed in [6], [9] and [10], so broad claims to it face serious novelty risk. The two mechanisms that appeared most promising in earlier drafts—health-verified progression and mid-recovery re-planning—are not implemented (Table X, objectives 7 and 8) and cannot support a claim until they are built and measured.

Table XI
Novelty Assessment by Mechanism

Mech.	Mechanism	Position against prior art	Implemented here	Novelty
A	Live dependency graph	Well known: service maps, CMDB, eBPF/OTel-derived graphs. Here: in-memory DAG defined by seed, YAML or LLM, not auto-discovered	Yes (declared topology)	Low
B	Failure-triggered blast radius	Known: CMDB impact analysis; BFS is a standard algorithm	Yes	Low
C	Dynamic recovery sequence from current topology	Claimed in [7]; ordered restoration in [9]; commercial ABR [6]	Partly (Kahn levels in API / MCP)	High risk for broad claims
D	Dependency-constrained execution with health checks	Known in orchestration (readiness, success conditions); close to [7], [10] when combined with a graph	No (timer-driven)	Not claimable
E	Risk classification with approval gate	Known: change-management gates, StackStorm / Rundeck approvals; testnet signing is an implementation choice	Yes (token not verified)	Low
F	Recovery audit trail	Standard operational requirement	Partly (in memory)	Low
G	Mid-recovery re-planning on failed check	Most promising candidate; requires true re-planning from the updated graph	No	Not claimable until built and measured
H	Atomic topology mount with cycle rejection and rollback	Circular dependencies are handled in [10]; mount-time rollback not found in the references reviewed	Yes (Python, 1 / 1 test)	Low to moderate; search needed

B. Inventive Step

A person skilled in site-reliability automation would probably regard the combination of Kahn ordering, an approval gate and an audit log as an obvious aggregation of known techniques. That view could change only if a surprising technical effect were demonstrated. The present evidence—zero ordering violations against 4.18–16.02 for naive orderings—shows the correctness of a standard algorithm rather than an unexpected effect, and the browser plan generator does not yet realise it end to end.

C. Enablement and Eligibility

The public repository and the deterministic TypeScript core give an enabling disclosure of the graph and gating logic. In India, section 3(k) of the Patents Act, 1970 excludes computer programmes *per se*, mathematical methods and algorithms [16], and the guidelines for computer-related inventions direct examiners to look for a technical effect or contribution [15]. The invention should therefore be framed as an improvement in the reliability and recovery behaviour of real distributed computing systems and supported by measured effect. Public release of the repository and of this paper may itself affect novelty, so counsel should be consulted before further disclosure.

D. Claim Concepts (Research Notes)

Concept 1 (method), restricted to elements present in the code: maintaining a directed dependency graph; rejecting a topology that contains a cycle and restoring the prior topology; declaring failure after a configurable number of consecutive missed probes; computing a blast-radius set by reverse traversal and a severity class from its size and depth; partitioning the affected set into topological levels; classifying recovery steps by risk; automatically executing low-risk steps; suspending a high-risk step until an approval token bound to the incident and step is received (binding is computed on the web path; server-

side verification of it is not implemented); and recording each transition. *Dependent concepts:* severity-dependent gating, atomic topology replacement with rollback, and declarative playbooks without raw shell strings. *Reserved until implemented and measured:* weighted edges, health-verified progression, mid-recovery re-planning, approval timeout with safe abort, rollback of a partially applied sequence, and threshold approval. Final claim language must be drafted by qualified patent counsel.

IX. Security Considerations

Autonomous recovery enlarges the attack surface. The principal threats are compromised automation credentials running destructive playbooks; dependency-graph poisoning that inverts the recovery order; approval bypass or token replay; audit tampering; recovery-loop attacks that repeatedly restart healthy services; monitoring spoofing that fabricates failures; and ransomware scenarios in which the recovery system itself becomes the vector. Our measurements make two exposures concrete: the Python verifier accepts arbitrary well-formed strings (Table VII), so approval is not yet non-repudiable, and the audit log is mutable in-process memory. Controls required before production use are least-privilege execution identities, signed and versioned playbooks, wallet-signed typed-data approvals verified server-side with nonce and expiry, append-only audit storage anchored through the existing vault contract, graph-integrity checks, circuit breakers on repeated failure or unexpected blast-radius growth, and threshold approval for the highest-risk classes. A blockchain approval path would provide non-repudiation of human decisions but does not by itself protect the execution plane.

X. Intellectual-Property Roadmap (Indian Context)

Table XII proposes a staged route to a provisional filing. Counsel experienced in Indian Patent Office practice on computer-related inventions should be retained throughout.

Table XII
Staged Route to a Provisional Filing

#	Action	Output
1	Freeze the disclosure: architecture, commit hash (6d459f6 or later), experimental protocol, inventor contributions	Signed disclosure record
2	Close implementation gaps in Table X (objectives 3, 7, 8, 6, 9)	Tagged release with tests
3	Formal prior-art search (USPTO, EPO, WIPO, IPO), including the Bank of America family and AIOps remediation patents	Search report
4	Written patentability opinion on residual novelty of narrow control claims	Counsel opinion
5	Provisional specification stressing the implemented, measured control logic	Filed provisional
6	Controlled experiments per Section VII-C; complete specification or PCT within the priority year	Complete specification / PCT

XI. Conclusion

Horizon's engine code implements the core graph operations correctly and efficiently. Blast-radius precision and recall were 1.00 across 1,007 incidents; Kahn ordering produced no dependency violations where breadth-first and alphabetical orderings produced 4.18 and 16.02 per incident; and the TypeScript engine ordered a 10,000-node graph in 13.9 ms. The Python engine's probe window, cycle rejection and gated playbooks behave as specified, though its blast-radius routine is quadratic.

The audit is equally clear about what the prototype does not yet do. Of 15 design objectives, five are fully met, five partly met and five not met: the browser plan generator does not use the topological order it computes, progression is timer-driven rather than health-verified, no mid-recovery re-planning exists, approval tokens are not cryptographically verified, and MTTR benefits are unmeasured. The most defensible technical ground remains the combination of live topological sequencing, mandatory approval for high-risk steps and health-verified progression, but the last element must first be built, and even then the combination is contested by existing patents and commercial systems. We therefore recommend closing the gaps in Section VII-B, running the four-baseline evaluation, and seeking counsel-reviewed claims that steer around US 12 556 441 B1 before any filing decision.

References

[1] L. Zhang *et al.*, "A survey of AIOps for failure management in the era of large language models," arXiv:2406.11213, 2024.

[2] K. Dichev, H. Jordan, K. Tovletoglou, T. Heller, D. S. Nikolopoulos, G. Karakonstantis, and C. Gillan, "Dependency-aware rollback and checkpoint-restart for distributed task-based runtimes," arXiv:1705.10208, 2017.

[3] "Autonomous incident resolution at hyperscale: An agentic AI architecture for network operations," arXiv preprint, 2026.

[4] S. K. Vytla, "AUTO-OPS: Predictive incident detection and self-healing remediation for cloud-native enterprise platforms," *Eng., Technol. Appl. Sci. Res.*, 2026.

[5] StackStorm, Rundeck, Kubernetes and enterprise AIOps vendor documentation (public project and vendor sources), 2015–2026.

[6] Rubrik, "Autonomous Business Recovery," product and technical documentation, 2026.

[7] Bank of America Corp., "System and method for computing infrastructure and operational restoral through automated dependency mapping," U.S. Patent 12 556 441 B1, 2026.

[8] "Techniques for automated service monitoring and remediation in a distributed computing system," U.S. Patent Appl. Publ. 2023/0205657 A1, 2023.

[9] "Automatically generating system restoration order for network recovery," U.S. Patent 8 667 096 B2.

[10] "Avoidance of dependency issues in network-based service startup workflows," U.S. Patent 9 183 092 B1.

[11] "Recovery segment identification in a computing infrastructure," U.S. Patent 7 490 265.

[12] "Systems and methods for automatic discovery of network software relationships," U.S. Patent Appl. Publ. 2010/0223609 A1.

[13] A. B. Kahn, "Topological sorting of large networks," *Commun. ACM*, vol. 5, no. 11, pp. 558–562, Nov. 1962.

[14] Precise-Goals, "Horizon: Autonomous infrastructure recovery platform," GitHub repository, commit 6d459f6, Nov. 2026. [Online]. Available: https://github.com/Precise-Goals/horizon

[15] Office of the Controller General of Patents, Designs and Trade Marks, India, "Guidelines for examination of computer related inventions (CRIs)," 2017.

[16] The Patents Act, 1970 (India), sec. 3(k).